```cpp
#include <Arduino.h>
#include <math.h>
#include "driver/gpio.h"

// ========================================================
// JY901S IMU
// ========================================================

HardwareSerial IMU_Serial(2);

#define IMU_RX_PIN 16
#define IMU_TX_PIN 17
#define IMU_BAUDRATE 9600

// true  = IMU X-axis is robot forward axis
// false = IMU Y-axis is robot forward axis
const bool IMU_FORWARD_AXIS_IS_X = false;

// ========================================================
// ENCODER PINS
// ========================================================

#define LEFT_ENCODER_A  32
#define LEFT_ENCODER_B  33

#define RIGHT_ENCODER_A 25
#define RIGHT_ENCODER_B 26

// ========================================================
// ENCODER CONFIGURATION
// ========================================================

const float WHEEL_RADIUS = 0.05f;
const float WHEEL_CIRCUMFERENCE = 2.0f * PI * WHEEL_RADIUS;

// Must represent rising edges of channel A per revolution
const float ENCODER_PPR = 2500.0f;

// Your tested encoder signs
const int LEFT_ENCODER_SIGN = -1;
const int RIGHT_ENCODER_SIGN = 1;

// ========================================================
```

```
// ENCODER COUNTERS
// ========================================================

volatile long leftEncoderCount = 0;
volatile long rightEncoderCount = 0;

portMUX_TYPE encoderMux =
    portMUX_INITIALIZER_UNLOCKED;

// ========================================================
// ENCODER DISTANCE VARIABLES
// ========================================================

float leftDistance = 0.0f;
float rightDistance = 0.0f;

float encoderDistance = 0.0f;
float encoderVelocity = 0.0f;

float previousLeftDistance = 0.0f;
float previousRightDistance = 0.0f;

float previousEncoderDistance = 0.0f;

// ========================================================
// ENCODER MOVING AVERAGE
// ========================================================

// Moving average is applied to distance movement increments,
// not directly to cumulative absolute position.

const int ENCODER_AVERAGE_WINDOW = 3;

float encoderMovementBuffer[ENCODER_AVERAGE_WINDOW] = {
  0.0f,
  0.0f,
  0.0f
};

int encoderBufferIndex = 0;
int encoderBufferCount = 0;

// ========================================================
```

```
// IMU VARIABLES
// ======================================================

// Raw acceleration values in m/s^2
float accelX = 0.0f;
float accelY = 0.0f;
float accelZ = 0.0f;

// Positive horizontal acceleration magnitude
float totalAcceleration = 0.0f;

// Signed acceleration along robot's forward axis
float imuLongitudinalAcceleration = 0.0f;

// IMU velocity and distance
float imuVelocity = 0.0f;
float imuDistance = 0.0f;

// Signed IMU distance according to encoder direction
float signedIMUDistance = 0.0f;

// ======================================================
// IMU SETTINGS
// ======================================================

const float GRAVITY = 9.80665f;

// Acceleration threshold
const float ACCELERATION_THRESHOLD = 1.5f;
const float MAX_VALID_ACCELERATION = 3.0f;

// Velocity gate
const float VELOCITY_GATE_THRESHOLD = 0.05f;

// Stop velocity after 3 seconds below acceleration threshold
const unsigned long IMU_STOP_TIMEOUT = 1500;

unsigned long lastIMUMovementTime = 0;
unsigned long previousIMUTime = 0;

// ======================================================
// DIRECTION
// ======================================================
```

```
enum Direction {
  STOPPED,
  FORWARD,
  BACKWARD,
  LEFT,
  RIGHT
};

Direction currentDirection = STOPPED;

// +1 for forward/left/right
// -1 for backward
//  0 for stopped or turning without forward displacement
int movementSign = 0;

// ======================================================
// KALMAN FILTER STATE
// ======================================================

// x[0] = position in metres
// x[1] = velocity in metres/second

float positionEstimate = 0.0f;
float velocityEstimate = 0.0f;

// Covariance matrix P
float P00 = 1.0f;
float P01 = 0.0f;
float P10 = 0.0f;
float P11 = 1.0f;

// ======================================================
// KALMAN FILTER TUNING
// ======================================================

const float ACCELERATION_NOISE = 1.0f;
const float ENCODER_POSITION_NOISE = 0.0001f;
const float ENCODER_VELOCITY_NOISE = 0.01f;

// ======================================================
// KALMAN INITIALIZATION
// ======================================================
```

```
void initializeKalman(float initialPosition) {
  positionEstimate = initialPosition;
  velocityEstimate = 0.0f;

  P00 = 1.0f;
  P01 = 0.0f;
  P10 = 0.0f;
  P11 = 1.0f;
}

// ========================================================
// KALMAN PREDICTION
// ========================================================

void kalmanPredict(float acceleration, float dt) {
  if (dt <= 0.0f || dt > 1.0f) {
    return;
  }

  // State prediction
  positionEstimate =
      positionEstimate +
      velocityEstimate * dt +
      0.5f * acceleration * dt * dt;

  velocityEstimate =
      velocityEstimate +
      acceleration * dt;

  // Process noise
  float dt2 = dt * dt;
  float dt3 = dt2 * dt;
  float dt4 = dt2 * dt2;

  float qPosition =
      0.25f * dt4 * ACCELERATION_NOISE;

  float qVelocity =
      dt2 * ACCELERATION_NOISE;

  float qPositionVelocity =
      0.5f * dt3 * ACCELERATION_NOISE;
```

```
  // Covariance prediction
  float newP00 =
      P00 +
      dt * (P01 + P10) +
      dt2 * P11 +
      qPosition;

  float newP01 =
      P01 +
      dt * P11 +
      qPositionVelocity;

  float newP10 =
      P10 +
      dt * P11 +
      qPositionVelocity;

  float newP11 =
      P11 +
      qVelocity;

  P00 = newP00;
  P01 = newP01;
  P10 = newP10;
  P11 = newP11;
}

// ======================================================
// KALMAN POSITION UPDATE
// ======================================================

void kalmanUpdatePosition(float measuredPosition) {
  float innovation =
      measuredPosition - positionEstimate;

  float innovationCovariance =
      P00 + ENCODER_POSITION_NOISE;

  if (innovationCovariance <= 0.0f) {
    return;
  }
```

```
  float K0 = P00 / innovationCovariance;
  float K1 = P10 / innovationCovariance;

  positionEstimate =
      positionEstimate + K0 * innovation;

  velocityEstimate =
      velocityEstimate + K1 * innovation;

  float oldP00 = P00;
  float oldP01 = P01;

  P00 = (1.0f - K0) * oldP00;
  P01 = (1.0f - K0) * oldP01;

  P10 = P10 - K1 * oldP00;
  P11 = P11 - K1 * oldP01;

  // Keep covariance symmetric
  P10 = P01;
}

// ========================================================
// KALMAN VELOCITY UPDATE
// ========================================================

void kalmanUpdateVelocity(float measuredVelocity) {
  float innovation =
      measuredVelocity - velocityEstimate;

  float innovationCovariance =
      P11 + ENCODER_VELOCITY_NOISE;

  if (innovationCovariance <= 0.0f) {
    return;
  }

  float K0 = P01 / innovationCovariance;
  float K1 = P11 / innovationCovariance;

  positionEstimate =
      positionEstimate + K0 * innovation;
```

```
  velocityEstimate =
      velocityEstimate + K1 * innovation;

  float oldP00 = P00;
  float oldP01 = P01;
  float oldP10 = P10;
  float oldP11 = P11;

  // Covariance update
  P00 = oldP00 - K0 * oldP10;
  P01 = oldP01 - K0 * oldP11;
  P10 = oldP10 - K1 * oldP10;
  P11 = oldP11 - K1 * oldP11;

  // Keep covariance symmetric
  P10 = P01;
}

// ========================================================
// COMPLETE KALMAN UPDATE
// ========================================================

void updateKalman(
    float signedAcceleration,
    float measuredPosition,
    float measuredVelocity,
    float dt
) {
  // 1. Predict using signed IMU acceleration
  kalmanPredict(
      signedAcceleration,
      dt
  );

  // 2. Correct position using encoder position
  kalmanUpdatePosition(
      measuredPosition
  );

  // 3. Correct velocity using encoder velocity
  kalmanUpdateVelocity(
      measuredVelocity
  );
```

```
}

// ========================================================
// LEFT ENCODER ISR
// ========================================================

void IRAM_ATTR leftEncoderISR() {
  int channelB =
      gpio_get_level((gpio_num_t)LEFT_ENCODER_B);

  portENTER_CRITICAL_ISR(&encoderMux);

  if (channelB == LOW) {
    leftEncoderCount++;
  } else {
    leftEncoderCount--;
  }

  portEXIT_CRITICAL_ISR(&encoderMux);
}

// ========================================================
// RIGHT ENCODER ISR
// ========================================================

void IRAM_ATTR rightEncoderISR() {
  int channelB =
      gpio_get_level((gpio_num_t)RIGHT_ENCODER_B);

  portENTER_CRITICAL_ISR(&encoderMux);

  if (channelB == LOW) {
    rightEncoderCount++;
  } else {
    rightEncoderCount--;
  }

  portEXIT_CRITICAL_ISR(&encoderMux);
}

// ========================================================
// GET DIRECTION TEXT
// ========================================================
```

```
const char* getDirectionText() {
  switch (currentDirection) {
    case FORWARD:
      return "FORWARD";

    case BACKWARD:
      return "BACKWARD";

    case LEFT:
      return "RIGHT";

    case RIGHT:
      return "LEFT";

    default:
      return "STOPPED";
  }
}

// ======================================================
// DETERMINE DIRECTION
// ======================================================

void determineDirection(
    float leftMovement,
    float rightMovement
) {
  const float MOVEMENT_EPSILON = 0.000001f;

  bool leftForward =
      leftMovement > MOVEMENT_EPSILON;

  bool leftBackward =
      leftMovement < -MOVEMENT_EPSILON;

  bool rightForward =
      rightMovement > MOVEMENT_EPSILON;

  bool rightBackward =
      rightMovement < -MOVEMENT_EPSILON;

  bool leftStopped =
```

```
      fabs(leftMovement) <= MOVEMENT_EPSILON;

  bool rightStopped =
      fabs(rightMovement) <= MOVEMENT_EPSILON;

  // Both positive = FORWARD
  if (leftForward && rightForward) {
    currentDirection = FORWARD;
    movementSign = 1;
  }

  // Both negative = BACKWARD
  else if (leftBackward && rightBackward) {
    currentDirection = BACKWARD;
    movementSign = -1;
  }

  // Right forward + left backward = RIGHT
  else if (rightForward && leftBackward) {
    currentDirection = RIGHT;
    movementSign = 1;
  }

  // Left forward + right backward = LEFT
  else if (leftForward && rightBackward) {
    currentDirection = LEFT;
    movementSign = 1;
  }

  // Both stopped = STOPPED
  else if (leftStopped && rightStopped) {
    currentDirection = STOPPED;
    movementSign = 0;
  }

  // If only one wheel moves, keep previous direction.
}

// ========================================================
// UPDATE ENCODER DISTANCE
// ========================================================

void updateEncoderDistance(float dt) {
```

```
long leftCount;
long rightCount;

portENTER_CRITICAL(&encoderMux);

leftCount = leftEncoderCount;
rightCount = rightEncoderCount;

portEXIT_CRITICAL(&encoderMux);

// Convert counts to revolutions
float leftRevolutions =
    ((float)leftCount / ENCODER_PPR) *
    LEFT_ENCODER_SIGN;

float rightRevolutions =
    ((float)rightCount / ENCODER_PPR) *
    RIGHT_ENCODER_SIGN;

// Convert revolutions to distance
float newLeftDistance =
    leftRevolutions * WHEEL_CIRCUMFERENCE;

float newRightDistance =
    rightRevolutions * WHEEL_CIRCUMFERENCE;

// Distance movement during this interval
float leftMovement =
    newLeftDistance - previousLeftDistance;

float rightMovement =
    newRightDistance - previousRightDistance;

previousLeftDistance = newLeftDistance;
previousRightDistance = newRightDistance;

leftDistance = newLeftDistance;
rightDistance = newRightDistance;

// Determine direction from corrected encoder movements
determineDirection(
    leftMovement,
    rightMovement
```

```
);

// Average the two wheel movements
float currentMovement =
    (leftMovement + rightMovement) / 2.0f;

// ---------------------------------------------------
// VELOCITY GATE
// ---------------------------------------------------

bool velocityGateOpen =
    fabs(imuVelocity) >= VELOCITY_GATE_THRESHOLD;

// Close gate if no IMU movement was detected recently
if (millis() - lastIMUMovementTime > IMU_STOP_TIMEOUT) {
  velocityGateOpen = false;
}

/*
  Apply moving average to movement increments.

  This avoids averaging cumulative absolute positions,
  which would cause a delayed/distorted position.
*/

if (velocityGateOpen) {
  encoderMovementBuffer[encoderBufferIndex] =
      currentMovement;

  encoderBufferIndex =
      (encoderBufferIndex + 1) % ENCODER_AVERAGE_WINDOW;

  if (encoderBufferCount < ENCODER_AVERAGE_WINDOW) {
    encoderBufferCount++;
  }

  float movementSum = 0.0f;

  for (int i = 0; i < encoderBufferCount; i++) {
    movementSum += encoderMovementBuffer[i];
  }

  float averageMovement =
```

```
        movementSum / (float)encoderBufferCount;

    encoderDistance += averageMovement;
  }

  // Calculate encoder velocity
  float positionDifference =
      encoderDistance - previousEncoderDistance;

  if (dt > 0.0f) {
    encoderVelocity =
        positionDifference / dt;
  } else {
    encoderVelocity = 0.0f;
  }

  previousEncoderDistance = encoderDistance;
}

// ========================================================
// PROCESS JY901S ACCELERATION PACKET
// ========================================================

uint8_t imuPacket[11];
int imuPacketIndex = 0;

void processIMUByte(uint8_t incomingByte) {
  // Wait for packet header
  if (imuPacketIndex == 0) {
    if (incomingByte != 0x55) {
      return;
    }
  }

  imuPacket[imuPacketIndex] =
      incomingByte;

  imuPacketIndex++;

  if (imuPacketIndex < 11) {
    return;
  }
```

```
imuPacketIndex = 0;

// 0x51 = acceleration packet
if (imuPacket[1] != 0x51) {
  return;
}

// Checksum
uint8_t checksum = 0;

for (int i = 0; i < 10; i++) {
  checksum += imuPacket[i];
}

if (checksum != imuPacket[10]) {
  return;
}

int16_t rawX =
    (int16_t)((imuPacket[3] << 8) | imuPacket[2]);

int16_t rawY =
    (int16_t)((imuPacket[5] << 8) | imuPacket[4]);

int16_t rawZ =
    (int16_t)((imuPacket[7] << 8) | imuPacket[6]);

// JY901S +/-16g conversion
float rawAccelX =
    ((float)rawX / 32768.0f) *
    16.0f *
    GRAVITY;

float rawAccelY =
    ((float)rawY / 32768.0f) *
    16.0f *
    GRAVITY;

float rawAccelZ =
    ((float)rawZ / 32768.0f) *
    16.0f *
    GRAVITY;
```

```
  accelX = rawAccelX;
  accelY = rawAccelY;
  accelZ = rawAccelZ;

  // ====================================================
  // TOTAL HORIZONTAL ACCELERATION
  // ====================================================

  // As requested, ignore Z-axis
  totalAcceleration =
      sqrt(
          (accelX * accelX) +
          (accelY * accelY)
      );

  // ====================================================
  // SIGNED LONGITUDINAL ACCELERATION
  // ====================================================

  if (IMU_FORWARD_AXIS_IS_X) {
    imuLongitudinalAcceleration = accelX;
  } else {
    imuLongitudinalAcceleration = accelY;
  }

  // Remove IMU noise below the selected threshold.
  // Remove small IMU noise and reject unusually high acceleration
if (totalAcceleration < ACCELERATION_THRESHOLD ||
    totalAcceleration > MAX_VALID_ACCELERATION) {

  totalAcceleration = 0.0f;
  imuLongitudinalAcceleration = 0.0f;
}

  // ====================================================
  // IMU VELOCITY AND DISTANCE
  // ====================================================

  unsigned long currentTime = millis();

  if (previousIMUTime == 0) {
    previousIMUTime = currentTime;
    return;
```

```
}

float dt =
    (currentTime - previousIMUTime) / 1000.0f;

previousIMUTime = currentTime;

if (dt <= 0.0f || dt > 1.0f) {
  return;
}

/*
  Use the kinematic formulas.

  v = u + a*t
  s = u*t + 0.5*a*t^2

  IMU velocity is signed according to direction.
  IMU distance is accumulated with the same sign.
*/

if (totalAcceleration >= ACCELERATION_THRESHOLD) {
  lastIMUMovementTime = currentTime;

  float signedAcceleration =
      imuLongitudinalAcceleration;

  float previousVelocity =
      imuVelocity;

  // Velocity formula
  imuVelocity =
      previousVelocity +
      signedAcceleration * dt;

  // Displacement formula
  float displacement =
      previousVelocity * dt +
      0.5f * signedAcceleration * dt * dt;

  // Keep the separate IMU distance signed
  imuDistance += displacement;
```

```
    // Same distance value for display/fusion reference
    signedIMUDistance = imuDistance;
  } else {
    // Stop velocity when acceleration is absent
    if (currentTime - lastIMUMovementTime > IMU_STOP_TIMEOUT) {
      imuVelocity = 0.0f;
    }
  }
}

// ========================================================
// KALMAN FUSION
// ========================================================

float fusedDistance = 0.0f;

// ========================================================
// POSITION PID CONTROLLER
// ========================================================

// The existing fusedDistance remains untouched.
// PID coordinate system: start = -5 m, target = 0 m.
// Since fusedDistance is zero at the sensor origin, the PID
// position is shifted by -5 m.
const float PID_START_POSITION = -5.0f;
const float PID_TARGET_POSITION = 0.0f;

// Initial PID gains. These are starting values for tuning;
// they cannot be considered physically tuned without the motors.
const float PID_KP = 30.0f;
const float PID_KI = 0.5f;
const float PID_KD = 10.0f;

// PWM command limits. Positive = forward, negative = backward.
const float PID_MAX_PWM = 255.0f;

// Prevent integral windup.
const float PID_INTEGRAL_LIMIT = 50.0f;

// Consider the target reached when position is close enough
// and the estimated velocity is sufficiently small.
const float PID_POSITION_TOLERANCE = 0.02f;
const float PID_VELOCITY_TOLERANCE = 0.05f;
```

```
float pidPosition = PID_START_POSITION;
float pidError = 0.0f;
float pidPreviousError = 0.0f;
float pidIntegral = 0.0f;
float pidP = 0.0f;
float pidI = 0.0f;
float pidD = 0.0f;
float pidOutput = 0.0f;
float pidPWM = 0.0f;

const char* pidDirection = "STOP";
bool pidTargetReached = false;

void updatePositionPID(float dt) {
  if (dt <= 0.0f) {
    return;
  }

  // Convert the existing fused sensor coordinate into the
  // required PID coordinate: -5 m at start and 0 m at target.
  pidPosition = fusedDistance + PID_START_POSITION;

  // Position error: positive means forward; negative means backward.
  pidError = PID_TARGET_POSITION - pidPosition;

  // Target is considered reached only when both position and velocity
  // are within tolerance.
  pidTargetReached =
      (fabs(pidError) <= PID_POSITION_TOLERANCE) &&
      (fabs(velocityEstimate) <= PID_VELOCITY_TOLERANCE);

  if (pidTargetReached) {
    pidIntegral = 0.0f;
    pidP = 0.0f;
    pidI = 0.0f;
    pidD = 0.0f;
    pidOutput = 0.0f;
    pidPWM = 0.0f;
    pidDirection = "STOP";
    pidPreviousError = pidError;
    return;
  }
```

```
  // Proportional term.
  pidP = PID_KP * pidError;

  // Integral term with anti-windup limit.
  pidIntegral += pidError * dt;
  pidIntegral = constrain(
      pidIntegral,
      -PID_INTEGRAL_LIMIT,
      PID_INTEGRAL_LIMIT
  );
  pidI = PID_KI * pidIntegral;

  // Derivative term from the Kalman velocity estimate.
  // error = target - position, so d(error)/dt = -velocity.
  pidD = -PID_KD * velocityEstimate;

  // Total signed PID command.
  pidOutput = pidP + pidI + pidD;

  // Limit command to the PWM range.
  pidOutput = constrain(
      pidOutput,
      -PID_MAX_PWM,
      PID_MAX_PWM
  );

  // Convert signed command into direction + PWM magnitude.
  if (pidOutput > 0.0f) {
    pidDirection = "FORWARD";
    pidPWM = pidOutput;
  }
  else if (pidOutput < 0.0f) {
    pidDirection = "BACKWARD";
    pidPWM = -pidOutput;
  }
  else {
    pidDirection = "STOP";
    pidPWM = 0.0f;
  }

  pidPreviousError = pidError;
}
```

```
void updateFusion(float dt) {
  /*
    The Kalman filter uses:

    - Signed IMU longitudinal acceleration for prediction
    - Encoder distance for position correction
    - Encoder velocity for velocity correction

    The separately calculated IMU distance is printed
    as a diagnostic distance.
  */

  updateKalman(
      imuLongitudinalAcceleration,
      encoderDistance,
      encoderVelocity,
      dt
  );

  fusedDistance = positionEstimate;
}

// ============================================================
// SETUP
// ============================================================

void setup() {
  Serial.begin(115200);

  // Start JY901S serial
  IMU_Serial.begin(
      IMU_BAUDRATE,
      SERIAL_8N1,
      IMU_RX_PIN,
      IMU_TX_PIN
  );

  // Encoder pins
  pinMode(LEFT_ENCODER_A, INPUT_PULLUP);
```

```
  pinMode(LEFT_ENCODER_B, INPUT_PULLUP);

  pinMode(RIGHT_ENCODER_A, INPUT_PULLUP);
  pinMode(RIGHT_ENCODER_B, INPUT_PULLUP);

  // Encoder interrupts
  attachInterrupt(
      digitalPinToInterrupt(LEFT_ENCODER_A),
      leftEncoderISR,
      RISING
  );

  attachInterrupt(
      digitalPinToInterrupt(RIGHT_ENCODER_A),
      rightEncoderISR,
      RISING
  );

  initializeKalman(0.0f);

  lastIMUMovementTime = millis();

  Serial.println("Distance fusion started");
}

// ============================================================
// LOOP
// ============================================================

unsigned long lastUpdateTime = 0;

void loop() {
  // Read all available IMU bytes
  while (IMU_Serial.available()) {
    uint8_t incomingByte =
        IMU_Serial.read();

    processIMUByte(incomingByte);
  }

  unsigned long currentTime = millis();

  // Main update every 100 ms
```

```
if (currentTime - lastUpdateTime >= 100) {
  float dt =
      (currentTime - lastUpdateTime) / 1000.0f;

  lastUpdateTime = currentTime;

  // Update encoder distance
  updateEncoderDistance(dt);

  // Update Kalman fusion
  updateFusion(dt);

  // Update position PID (calculation only; no motor output yet)
  updatePositionPID(dt);

  // ================================================
  // ONLY REQUIRED SERIAL OUTPUT
  // ================================================

  Serial.print("Acceleration: ");
  Serial.print(totalAcceleration, 4);
  Serial.print(" m/s^2");

  Serial.print(" | Velocity: ");
  Serial.print(imuVelocity, 4);
  Serial.print(" m/s");

  Serial.print(" | Direction: ");
  Serial.print(getDirectionText());

  Serial.print(" | Fused Distance: ");
  Serial.print(fusedDistance, 4);
  Serial.println(" m");

  Serial.print("PID Position: ");
  Serial.print(pidPosition, 4);
  Serial.print(" m | Target: ");
  Serial.print(PID_TARGET_POSITION, 4);
  Serial.print(" m | Error: ");
  Serial.print(pidError, 4);

  Serial.print(" | PID Output: ");
  Serial.print(pidOutput, 4);
```

```
    Serial.print(" | PWM: ");
    Serial.print(pidPWM, 2);
    Serial.print(" | PID Direction: ");
    Serial.print(pidDirection);
    Serial.print(" | Target Reached: ");
    Serial.println(pidTargetReached ? "YES" : "NO");
  }
}
```